Slovak University of Technology in Bratislava
Faculty of Informatics and Information Technology
Study programme: Intelligent Software Systems

Deep Learning Methods for Computer Vision in Autonomous Systems

Research Proposal and Experiments

Bc. Richard Čerňanský
2025/2026

Thesis supervisor: Ing. Matej Halinkovič

Contents

1 Research Proposal **3**

1.1 Introduction . . . 3
1.2 Motivation . . . 3
1.3 Problem Statement . . . 4
1.3.1 Data Fusion . . . 4
1.3.2 Dynamic Environments . . . 4
1.3.3 Interaction Modeling . . . 4
1.3.4 Real-World Constraints . . . 4
1.4 Taxonomy of Trajectory Prediction Methods . . . 5
1.4.1 Physics-Based Methods . . . 5
1.4.2 Classic Machine Learning Methods . . . 5
1.4.3 Deep Learning Methods . . . 5
1.4.4 Reinforcement Learning Methods . . . 5
1.4.5 Modular vs. End-to-End approaches . . . 6
1.5 Analysis of the State of the Art . . . 7
1.5.1 Classical foundations . . . 7
1.5.2 Deep learning wave (2016–2019) . . . 8
1.5.3 2020+ breakthroughs: vectorized maps, transformers, and unified BEV 9
1.6 Datasets . . . 11
1.7 Evaluation Metrics . . . 11
1.8 Problem Elaboration . . . 12
1.9 Goals and Expected Outcomes . . . 14
1.10 Conclusion . . . 14
1.11 Plan for Three Semesters for DPI, DPII, DPIII . . . 14

2 Research and Experiments **17**

2.1 DP1 Experiments . . . 17
2.1.1 Related Work . . . 17
2.1.2 Setup . . . 18
2.1.3 Baseline: Single-Frame CenterPoint . . . 18

2.1.4 First Attempt: Recurrent Temporal Extension 18
2.1.5 Second Attempt: ReasonNet-Style Temporal Memory Bank 20
2.1.6 Quantitative Summary 24
2.1.7 Discussion 25
2.2 LiViP3D: Query-Based LiDAR–Camera Fusion for End-to-End Perception and Prediction 26
2.2.1 Related Work 26
2.2.2 Model Architecture 26
2.2.3 Experimental Setup 29
2.2.4 Results 29
2.2.5 Discussion 30

Chapter 1

Research Proposal

1.1 Introduction

Autonomous vehicles must not only see the road but also anticipate what happens next. Trajectory prediction estimates how cars, cyclists, and pedestrians are likely to move over the next few seconds. Done well, it turns a cautious robot into a considerate road user: the car plans smoother merges, yields earlier, and avoids last-second braking. The difficulty is that traffic is messy, intentions change, sensors are imperfect, and several futures can be plausible at once. Modern systems therefore combine cues from cameras, LiDAR, and maps to form a consistent view of the scene and predict multiple probability-scored futures rather than a single guess. This work explores practical ways to fuse inputs, model interactions between agents, and express uncertainty, aiming for predictions that are accurate, fast enough for on-board use, and easy to integrate into real-time planning.

1.2 Motivation

Real traffic is highly dynamic and unpredictable: other drivers may behave erratically, and sensors provide noisy or incomplete data. Diverse maneuvers, complex interactions, and uncertainty in sensory information make prediction challenging, yet solving these challenges is vital for improving AV safety. In addition, autonomous systems operate under strict constraints on latency and reliability [1]. Predictions must be computed in real time on embedded hardware and remain accurate over long horizons to be useful for high-speed driving. Industry recognizes the need for fast execution, uncertainty handling, and generalization in prediction models to ensure robust operation in varied conditions [2, 3, 4]. Reliable trajectory prediction is therefore a critical enabler for safe autonomous driving and motivates intensive research into methods that can forecast the multi-agent future under uncertainty and with practical efficiency.

1.3 Problem Statement

Trajectory prediction with multimodal sensor input presents several core challenges. Modern autonomous vehicles are equipped with cameras, LiDAR, radar, and HD maps, each providing complementary information about the environment [5]. Fusing these heterogeneous data streams is non-trivial: they have different formats, update rates, fields of view, and noise characteristics. A naive approach might bolt together separate modules for each modality, but this can lead to complex systems that are hard to scale or tune. The research problem is centered on how to design a deep learning model that integrates multi-sensor data into a coherent view of the scene, enabling better trajectory forecasts. Key challenges include:

1.3.1 Data Fusion

Combine cameras, LiDAR or radar, and HD maps into one scene view. Compare early, BEV or attention-based intermediate fusion, and late fusion. Prevent overload or overspecialization through learned alignment and robust training [5].

1.3.2 Dynamic Environments

Traffic is nonstationary and many futures are plausible. Aleatoric uncertainty means the same history can lead to different outcomes. Predict a calibrated set of socially compliant trajectories so planning can weigh alternatives in real time [5].

1.3.3 Interaction Modeling

Agent motions are coupled and constrained by lanes and rules. Use structured representations such as graphs and attention to capture agent to agent and agent to map relations. Poor interaction modeling leads to infeasible predictions [1].

1.3.4 Real-World Constraints

The system must run within an onboard real-time loop and scale to dense traffic. Balance model capacity with memory and compute efficiency to stay deployable, not only accurate [3, 1].

In summary, the problem is how to develop a deep learning-based trajectory prediction approach that fuses multimodal sensor data, deals with noisy dynamic environments, models agent interactions, and produces reliable, uncertainty-aware trajectory forecasts in real time.

1.4 Taxonomy of Trajectory Prediction Methods

The literature on trajectory forecasting for autonomous driving can be categorized into four broad families [5]:

1.4.1 Physics-Based Methods

These rely on kinematic or dynamic models of motion, using physical equations to extrapolate future positions. Examples include constant-velocity or constant-acceleration models (Single-Trajectory simulation), Kalman filters which estimate state with uncertainty, and Monte Carlo simulations that sample many possible motions. Physics-based approaches are simple and fast, but they struggle with complex maneuvers or sudden behavior changes since they lack contextual understanding. Some variants combine multiple motion models to handle different maneuvers (Multiple-Model methods).

1.4.2 Classic Machine Learning Methods

These methods learn motion patterns from data using statistical models, but without deep neural networks. Common approaches include Hidden Markov Models (HMMs), Gaussian Processes, Dynamic Bayesian Networks, and Gaussian Mixture Models. They can capture uncertainty to a degree (e.g., an HMM can model discrete maneuver states, Gaussian Processes provide a distribution over trajectories) and were popular before deep learning. However, their capacity to model very complex, high-dimensional scenarios is limited compared to modern deep nets.

1.4.3 Deep Learning Methods

Deep models now dominate trajectory prediction. RNNs and LSTMs capture temporal history, CNNs extract spatial cues from rasterized agents and maps, GNNs model agent to lane relations, and Transformers provide long-range, heterogeneous context with attention. Generative decoders (VAEs, GANs, normalizing flows, diffusion) yield multi-hypothesis futures with calibrated uncertainty. State-of-the-art systems pair a strong scene encoder with multi-modal decoders and explicit interaction and map context. Robustness comes from multimodal fusion: early fusion on raw or shallow features, intermediate fusion with cross-modal attention or learned BEV alignment, and late fusion via ensembling or confidence weighting. The goal is a probability-scored, socially compliant, map-consistent set of trajectories that runs in real time.

1.4.4 Reinforcement Learning Methods

These methods treat prediction as a sequential decision problem. Here, an agent learns a policy (via reward feedback) to mimic human driving behavior, effectively “planning” likely

trajectories. RL-based predictors can naturally handle multi-agent interactions by modeling how agents might react to each other, and can ensure physically feasible outputs through learned policy constraints. However, RL methods can be data-hungry and harder to train/stabilize, and historically have been more common in planning than in pure trajectory forecasting. They are an active research area for capturing game-theoretic interactions and long-horizon planning in prediction.

1.4.5 Modular vs. End-to-End approaches

Definitions. A *modular* stack decomposes the pipeline into perception → tracking → map reasoning → prediction (and planning), with typed interfaces (objects, lanes, lights) [3, 6, 7, 8]. An *end-to-end* (E2E) stack trains a unified model to map raw sensors directly to trajectories (or planner-friendly representations), optionally with intermediate supervision [9, 10, 11]. Recent works also explore *semi end-to-end* designs that share a unified BEV backbone and train multiple heads jointly for detection, mapping, and motion forecasting [10, 9].

Modular stacks — advantages

- **Interpretability & diagnosability:** intermediate outputs (detections, tracks, lanes) can be inspected, unit-tested, and bounded; faults can be isolated to a component. This aligns with safety engineering [4].
- **Contracted interfaces & graceful degradation:** planning can reason over explicit objects and lanes; fallbacks are easier when a component underperforms.
- **Deterministic budgeting:** latency, memory, and compute can be budgeted per module for real-time loops.

Modular stacks — limitations

- **Error compounding and information bottlenecks:** early discretization (NMS, class thresholds) discards uncertainty; downstream stages cannot recover missed cues [1].
- **Objective mismatch:** components are trained on proxy losses (e.g., mAP for detection) that do not directly optimize forecasting quality.
- **Complex integration cost:** many interfaces demand dataset- and domain-specific tuning.

End-to-end stacks — advantages

- **Joint optimization:** shared features couple perception and forecasting, reducing handoff losses and leveraging weak cues across modalities [9].

- **Robustness to intermediate errors:** the model can learn to bypass brittle interfaces and propagate uncertainty to the output distribution.
- **Simplicity of training target:** directly optimizes forecasting objectives (ADE/FDE, minADE/minFDE, mAP).

End-to-end stacks — limitations

- **Verification & certification:** opaque internals complicate assurance cases and safety sign-off [4].
- **Data & compute demand:** strong supervision and large-scale training are typically required; closed-loop stability must be validated.
- **Debuggability:** failure analysis is harder without explicit intermediate artifacts.

1.5 Analysis of the State of the Art

1.5.1 Classical foundations

Kalman filtering (KF)

The modern baseline for linear, Gaussian state-space estimation. It introduced recursive minimum-variance estimation and remains the conceptual backbone for belief over state in motion models. It predicts trajectories as a two step process of filtering (denoising current state from noisy measurements) and forecasting (rolling the dynamics forward and propagating covariance - gaussian). This approach is very fast, good for short-horizon extrapolation but the gaussian assumption makes predicting longer sequences (the predictions get progressively worse). [12]

Extended KF (EKF) and Unscented KF (UKF)

For nonlinear dynamics/observations, EKF linearizes models locally; UKF propagates sigma points through true nonlinearities to avoid Jacobians and reduce linearization error, often outperforming EKF in practice. [13]

Particle filtering (Sequential Monte Carlo)

Handles non-Gaussian, highly nonlinear tracking by representing the posterior with weighted samples (the bootstrap filter). This unlocked robust multi-modal belief tracking for maneuvering targets. [14]

Markov Models

A general-purpose probabilistic framework for intent/maneuver recognition and sequence modeling that influenced early behavior prediction pipelines. [15]

Social force models (crowds)

For pedestrian motion, "forces" encode collision avoidance and social conventions. While hand-crafted, they established interaction modeling as essential for human-centric forecasting. [16]

Gaussian Processes (GPs)

Nonparametric Bayesian regression with principled uncertainty. Used to learn motion patterns and priors with continuous trajectories. [17]

Physics-based models (CV, CA, CTRA)

Deterministic kinematic and dynamic models encode physics and agent constraints directly. Common choices include constant velocity, constant acceleration, and constant turn rate with acceleration (CTRA) for short horizon motion. Bicycle models enforce non-holonomic limits, curvature, jerk, and actuator bounds. These models are interpretable and data efficient, and they fit into MPC or act as priors or constraints within KF, UKF, or SMC fusion. They can be brittle under behavior shifts. Modern systems often add learned residuals or intent layers to capture unmodeled interactions. These assumptions still echo in today's neural encoders, decoders, and losses.

1.5.2 Deep learning wave (2016–2019)

Sequence RNNs (LSTM)

Social-LSTM jointly reasons across agents with a pooling mechanism for interactions, replacing hand-designed forces with learned social context. [18]

Diversity via generative modeling (GANs/CVAEs)

Social-GAN introduced adversarial training + variety loss to generate multiple socially plausible futures (multi-modality). CVAE-style decoders popularized latent sampling for diverse trajectory hypotheses. These models introduced a shift from hand-crafted dynamics to learned interaction encoders and explicitly multi-modal output heads, anticipating later multi-hypothesis metrics and leaderboards. Contemporary surveys track this pivot from physics-based to learning-based forecasting. [19]

1.5.3 2020+ breakthroughs: vectorized maps, transformers, and unified BEV

Vectorized road graphs & multi-hypothesis heads

VectorNet (CVPR 2020) Encodes lanes/agents as polylines, aggregating local-to-global vector features; seminal for representing HD maps natively. [6]

LaneGCN (ECCV 2020) Graph convolutions over lane graphs plus agent–lane interactions; strong accuracy/efficiency trade-offs. [20]

TNT / DenseTNT (2020–2021) Predicts a small set of likely endpoints/goals first, then generates goal-conditioned full trajectories. This aligns with endpoint-centric metrics (minFDE) and with how drivers commit to destinations (lane exit, stop line, merge point). [7, 8]

CoverNet (CVPR 2020) Discrete trajectory set coverage with calibrated probabilities; influential for classification-style multi-modal heads. [21]

Trajectron++ (ECCV 2020) Models multi-agent interactions explicitly and outputs probabilistic, multi-modal trajectories via a CVAE (latent-variable) decoder. Integrates map context and enables dynamic behavior modeling. It uses per node type Encoder LSTM and a Decoder LSTM inside the CVAE (per node type, shared) to roll out future trajectories given latent z and context. Popular codebase and benchmark. [22]

Transformers for motion forecasting

Wayformer (2022) Wayformer treats everything in the scene as tokens, agent history points, nearby agents, roadgraph polylines/lane elements, and traffic-light states, and projects them to a common embedding. It then explores early, late, and hierarchical fusion, all built from the same multi-head self-attention backbone. Early fusion concatenates all tokens and runs one encoder; late fusion encodes each modality separately and fuses later; hierarchical fusion mixes both. Efficiency comes from factorized attention (temporal to spatial) and latent queries that compress context, while the decoder produces multi-hypothesis trajectories (mixture outputs with confidences). The result is a simple, homogeneous "transformer-as-universal-fuser" that scales across datasets (WOMD/Argoverse) without hand-crafted fusion modules, and cleanly ablates what each modality adds. [3]

Scene Transformer / hierarchical & heterogeneous variants (2020–2022) These models encode agent↔agent and agent↔map relations with attention but introduce structure to keep

computation bounded in dense scenes. Typical designs run local self-attention within an agent's temporal tokens, cross-attention to a small neighborhood of other agents, and attention to relevant lane/map segments (often vectorized polylines). A hierarchy aggregates fine-grained tokens into coarser scene summaries (clusters, pooled lane segments) and then broadcasts context back to agents, enabling long-range interaction reasoning without quadratic cost. Relational biases (e.g., distance, heading, lane connectivity) are injected as edge features or attention biases. For output, these encoders pair with multi-modal heads (goal/endpoint-conditioned decoders, K-trajectory heads, or likelihood models) to produce diverse, probability-scored futures that respect lane topology and social constraints. [23]

Vision-centric, unified BEV perception-to-prediction

LSS (Lift-Splat-Shoot, ECCV 2020) Introduced efficient camera-only BEV via implicit 3D unprojection; later became a backbone for unified BEV stacks. [24]

FIERY (ICCV 2021) Predicts future instance segmentation and motion directly in BEV from monocular surround cameras, producing probabilistic futures convertible to trajectories, a key step toward vision-only forecasting. [10]

BEVerse (2022–2023) Unifies detection, mapping, and motion forecasting in a single multi-camera BEV representation, reducing error accumulation between modules and enabling joint optimization. [9]

MultiXNet (2020) Early end-to-end LiDAR-centric model combining detection and prediction with multi-class, multi-modal outputs; symbolic for perception-to-forecasting pipelines beyond two-stage stacks. [11]

ReasonNet ReasonNet–style modules are multi-step, learned reasoning controllers that operate over visual tokens (CNN/ViT features, region proposals, BEV cells). A small recurrent controller (MLP/LSTM/transformer) runs iterative attention over a visual memory, updates an internal state, and uses gating/termination to decide the number of steps, querying regions, composing evidence (relations, attributes, counts), and refining hypotheses before emitting an answer or structured prediction. The trade-off is extra latency and training stability; the payoff is stronger multi-hop inference than one-shot feed-forward heads, especially when decisions hinge on relational cues. [25]

The field converged on vectorized maps and attention for scalable scene reasoning, and on unified BEV encoders that fuse cameras (optionally LiDAR/radar) and jointly train detection, mapping, and forecasting. This improved sample efficiency, temporal consistency, and real-

time integration. Recent surveys and leaderboards reflect this differentiation (vector-graph vs. unified-BEV, and LiDAR+camera vs. camera-only).

1.6 Datasets

Research in trajectory prediction has been accelerated by large-scale autonomous driving datasets that provide rich multimodal sensor data and trajectory annotations. Some prominent datasets include:

- **KITTI (2012):** Early AV benchmark with stereo, LiDAR, and GPS/IMU, great for detection, tracking, and SLAM. It is small (150 scenes) and supports only short-horizon forecasting, so it's limited for learning social behavior. [26]
- **nuScenes (2020):** Large multimodal suite (6 cameras, 5 radars, 1 LiDAR, 360°) with 1,000 twenty-second scenes, 3D boxes, and HD maps. It added a prediction challenge ($\approx$ 6 s horizon) and is ideal for multi-sensor fusion and map-aware forecasting. [27]
- **Argoverse (2019):** Includes a Motion Forecasting set with 324k five-second scenarios and rich HD lane graphs (Miami/Pittsburgh). It popularized map-based prediction and uses ADE/FDE and Miss Rate; Argoverse 2 expands coverage. [28]
- **Waymo Open Motion (2021):** Very large-scale forecasting dataset (>100k twenty-second scenarios at 10 Hz) with HD maps and many interactive scenes. It introduced mAP and Miss Rate for multi-modal sets and is a key benchmark for vectorized/transformer models. [2]
- **Lyft Level 5:** Large urban dataset with synchronized LiDAR/cameras and HD maps, similar in format to nuScenes. Widely used for 3D detection/tracking and baseline motion forecasting with map context. [29]
- **ApolloScape:** Chinese urban driving data covering detection, segmentation, tracking, and mapping. Useful for perception and localization; less standardized for forecasting than nuScenes/Argoverse. [30]

1.7 Evaluation Metrics

Across these benchmarks, common metrics to evaluate trajectory prediction include [5, 2]:

- **Average Displacement Error (ADE):** the average Euclidean distance between the predicted trajectory and the ground-truth trajectory over all time steps. It measures overall prediction accuracy.

- **Final Displacement Error (FDE):** the Euclidean distance between the predicted final position (at the prediction horizon, e.g., 3 s or 6 s) and the true final position. This emphasizes endpoint accuracy (did the model correctly predict where the agent ends up?).
- **minADE and minFDE (multi-modal prediction):** for models that output K possible futures per agent, these take the error of the closest prediction to ground truth (assume the model's best hypothesis). minADE/FDE reflect the best-case performance of a probabilistic predictor.
- **Miss Rate (MR):** usually defined as the fraction of cases where none of the K predicted trajectories is within a certain distance (e.g., 2 m) of the ground truth at the final time. A lower miss rate means the model almost always has at least one prediction near the true outcome, which is important for safety.
- **Mean Average Precision (mAP):** introduced in Waymo's benchmark, mAP treats trajectory prediction as a detection problem in trajectory space. It evaluates the precision–recall curve of the model's predicted set of trajectories against ground truth, taking into account both spatial tolerance and assigned confidence scores. It effectively rewards methods that produce a set of diverse predictions that cover the ground truth while avoiding too many implausible guesses.
- **Other metrics:** Negative Log-Likelihood (NLL) for probabilistic methods (measuring the quality of the predicted probability distribution), and occasionally collision rate or overlap metrics to ensure predicted trajectories are physically feasible (not crashing into others or violating road constraints).

Using these metrics in combination provides a comprehensive evaluation: ADE/FDE for accuracy, minADE/minFDE and MR for multimodal coverage, and mAP for overall usefulness of the prediction set. The literature often reports all of them to compare methods. For instance, a method might achieve lower ADE than another (better average accuracy), but if its miss rate is high it means it sometimes totally misses possible outcomes, a trade-off the proposal's approach must consider.

1.8 Problem Elaboration

Based on the literature review, several gaps and research questions emerge. A few key research directions will guide this thesis:

RQ1: How can we improve long-horizon trajectory forecasting in dense traffic scenarios using multimodal inputs and multimodal output trajectories?

We frame this as an architectural design problem: compare scene representations (vectorized lane/agent graphs vs. unified BEV grids), choose the fusion stage/mechanism (early projections vs. intermediate cross-modal attention, gating, or learned BEV/3D alignment, vs. late confidence aggregation). Design the temporal–interaction core (factorized temporal/spatial attention or relation-typed GNN message passing) to stabilize long-horizon rollouts. Select a multi-hypothesis decoder (endpoint/goal-conditioned anchors with residual refinement), enforce feasibility, and integrate efficiency enablers to meet real-time budgets. Effectiveness will be judged by ADE/FDE, minADE/minFDE@K, Miss Rate, endpoint mAP, off-road/collision rates, calibration quality, and latency, targeting consistent gains at 5–10 s horizons while remaining deployable.

RQ2: How to ensure uncertainty-aware and socially acceptable predictions in urban scenarios using deep learning models?

We want to explore diverse, probability-calibrated sets of futures using probabilistic decoders discrete set heads (endpoint/anchor + residuals). Social compliance will be enforced through interaction encoders (graph/transformer over agents or other possible actors) and environment topology. Ablations vary decoder type, number of samples/K modes, interaction strength, and uncertainty estimators. Evaluation uses minADE/minFDE@K, endpoint mAP, Miss Rate, plus qualitative audits. The aim is a recipe balancing diversity, realism, and computation.

RQ3: With the chosen input modalities (e.g., cameras, LiDAR, and their BEV projections), which scene representations and training objectives yield reliable and compact trajectory prediction?

We will build different predictors with plug-in encoders (camera, LiDAR) and a modality-agnostic backbone and temporal cores (factorized attention, causal conv/RNN). Decoders will compare different endpoint heads (K-trajectory, probabilistic objectives). We will select the most compact setup by jointly optimizing ADE/FDE, minADE/minFDE@K, Miss Rate, endpoint mAP or latency/memory. By addressing these questions, the thesis will target gaps identified in current research: the need for better sensor fusion methods, the need for uncertainty-aware models that interact safely, and the need for solutions that remain reliable across varied conditions.

1.9 Goals and Expected Outcomes

- Develop a multimodal trajectory prediction model that fuses camera and LiDAR to generate real-time, probability-scored multi-horizon trajectories.
- Establish quantitative evaluation on public benchmarks targeting gains in ADE/FDE, minADE/minFDE@K, MR, mAP, and latency.
- Assess uncertainty and safety through probability calibration, failure-mode analysis, and risk-aware behavior audits.
- Demonstrate generalization across scenarios and datasets with limited degradation relative to in-domain results.
- Produce academic contributions and documentation, including a rigorous review, reproducible code, and a polished thesis manuscript.

1.10 Conclusion

This work aims to deliver a practical trajectory prediction framework that improves accuracy, coverage, and calibration while remaining deployable in real time. It also provides clear evidence and tooling that advance multimodal forecasting research and its safe integration into autonomous systems.

1.11 Plan for Three Semesters for DPI, DPII, DPIII

DPI — problem framing and first baseline

Objectives

- Define the main research questions and system boundaries.
- Prepare data, metrics, and a basic evaluation setup.
- Implement at least one simple baseline model.

Activities

- Data preparation and checks of inputs, labels, and metrics.
- Training and testing of the baseline model, basic error analysis.

Timeline (indicative)

- Weeks 1–4: Data setup, metrics, and minimal pipeline; first runs.
- Weeks 5–9: Baseline training and initial experiments.
- Weeks 10–14: Stabilisation of the pipeline and short summary.

Deliverables

- Short written report with baseline results and a working prototype.

DPII — model design and comparison of setups

Objectives

- Design and implement the main deep learning architecture.
- Compare several model variants (e.g., single-modal vs. multimodal).
- Analyse accuracy, robustness, and computational cost.

Activities

- Implementation of model variants under a common interface.
- Systematic experiments and basic hyperparameter tuning.

Timeline (indicative)

- Weeks 1–4: First integrated model in the pipeline; sanity checks.
- Weeks 5–9: Experiments with different model variants and settings.
- Weeks 10–14: Performance tuning and interim summary.

Deliverables

- Interim report with comparison of model setups and main findings.

DPIII — final experiments and thesis

Objectives

- Finalise the chosen model configuration.
- Run extended experiments and evaluate trade-offs in detail.
- Complete the written thesis and prepare the final presentation.

Activities

- Final training runs and evaluation on all selected scenarios.
- Preparation of tables, figures, and reproducible scripts.

Timeline (indicative)

- Weeks 1–4: Final experiments and robustness checks.
- Weeks 5–9: Detailed analysis and consolidation of results.
- Weeks 10–14: Writing, polishing, and defence preparation.

Deliverables

- Final thesis, code repository with experiments, and a demo for defence.

Diploma Project 1 (DP1) Assignment

In Diploma Project 1 you will analyse the current use of deep learning methods for computer vision in autonomous systems and identify a concrete yet broadly applicable research problem related to perception, scene understanding, or motion prediction [5]. You will focus on exploring and comparing alternative deep learning model configurations for single-sensor inputs (e.g. camera-only) and multimodal inputs (e.g. combinations of cameras or LiDAR) that are suitable for deployment in autonomous systems [3]. Based on a critical review of recent work on deep learning for autonomous driving, you will formulate clear research objectives, define the required input–output behaviour of the system, and specify suitable evaluation criteria, with particular emphasis on the trade-offs between prediction accuracy, robustness, and computational efficiency [1]. You will propose a conceptual design of one or more deep learning solutions that process realistic sensor data and support decision making in an autonomous system. You will implement an experimental prototype, select a representative dataset, and conduct systematic experiments to compare the proposed configurations with simple baselines, analysing both performance and resource requirements. Finally, you will document the methodology, results, and limitations, and summarise recommendations for further development and for integration of the proposed solution into larger software or research systems.

Chapter 2

Research and Experiments

2.1 DP1 Experiments

This section presents two experiments on temporal LiDAR detection on the nuScenes dataset [27]. The objective is twofold: to establish a strong single-frame baseline, and to investigate two families of temporal extensions that exploit the sequential nature of LiDAR sweeps. The first family is the convolutional-recurrent extension of PointPillars [31] popularised by LSTM Pillars [32] and TimePillars [33]. The second family is the attention-based memory bank introduced by ReasonNet [25], reimplemented on top of a pretrained CenterPoint detector [34]. Both families underperform the single-frame baseline: the first by a small and well-understood margin, the second by a catastrophic collapse traced to a failure of the attention mechanism. The negative results are diagnostic rather than terminal: they characterise the failure modes of temporal LiDAR-only detectors built on a frozen perception backbone. All experiments are conducted within the OpenPCDet framework [35], on the nuScenes train/val splits, on two NVIDIA GeForce RTX 4090 GPUs (24 GB VRAM each).

2.1.1 Related Work

PointPillars [31] encodes a LiDAR point cloud into a pseudo-image of vertical pillars and applies a 2D CNN backbone, making it the standard fast, single-frame LiDAR detector this section builds on. **CenterPoint** [34] replaces anchor-based detection with a center-based heatmap head predicted directly on the BEV feature map, avoiding hand-tuned anchor boxes and NMS-heavy matching; paired with a PointPillars encoder, it is the single-frame baseline used throughout this section.

LSTM Pillars [32] and **TimePillars** [33] both add a convolutional-recurrent cell after the pillar encoder to fuse BEV features across consecutive sweeps, with TimePillars additionally learning an ego-motion warping module to align past and present feature maps – the first family of temporal extensions reproduced here. **ReasonNet** [25] instead maintains a short- and long-term memory bank of past BEV features and reads from it via content-based attention, the design

underlying the second, attention-based extension; its attention-collapse failure mode is analysed here using the fix proposed for memory-based video segmentation by **STCN** [36]. All models are implemented in **OpenPCDet** [35], the open-source 3D detection framework used for every experiment in this section.

2.1.2 Setup

All models share the same input configuration. The LiDAR point cloud is cropped to the range $[-51.2, -51.2, -5.0, 51.2, 51.2, 3.0]$ m and voxelised with a pillar size of $(0.2, 0.2, 8.0)$ m, yielding a 512×512 BEV grid. The CenterPoint head receives features at stride 4 (BEV resolution 128×128 at 0.8 m per cell) and predicts the standard nuScenes outputs (heatmap, centre offset, height, 3D size, yaw, and 2D velocity) for ten classes. Evaluation follows the nuScenes protocol on the official validation split, reporting mean Average Precision (mAP), the nuScenes Detection Score (NDS), and the five true-positive error metrics (mATE, mASE, mAOE, mAVE, mAAE).

2.1.3 Baseline: Single-Frame CenterPoint

The baseline is a CenterPoint detector with a PointPillars encoder, trained on the nuScenes train split for 20 epochs with the default OpenPCDet recipe: AdamW with weight decay 0.01, one-cycle learning-rate schedule, and a batch size of 4 per GPU. The trained weights are used both as the reference single-frame model and as initialisation for the two temporal variants discussed below.

On the nuScenes validation split, the baseline reaches an NDS of 0.6171 and an mAP of 0.5146, which is consistent with the publicly reported numbers for the OpenPCDet implementation of CenterPoint and slightly above the original PointPillars baseline. The per-class and true-positive metrics are reported in Tables 2.1 and 2.2, alongside the two temporal variants.

2.1.4 First Attempt: Recurrent Temporal Extension

Motivation and Pipeline

LSTM Pillars [32] and TimePillars [33] both show that a convolutional recurrent module placed after the pillar encoder of a single-frame detector can recover temporal cues from consecutive LiDAR sweeps. The first experiment reproduces this idea on nuScenes. The proposed model, denoted *TemporalPointPillar*, processes the N most recent LiDAR keyframes through the shared (pretrained) Pillar Feature Net to obtain a sequence of BEV feature maps $\{F_1, \ldots, F_N\}$, and then applies a convolutional recurrent cell:

$$H_t = \text{RecurrentCell}(F_t,\ H_{t-1}), \qquad H_0 = 0.$$

The final hidden state H_N is passed through the (pretrained) 2D backbone and CenterPoint head exactly as in the single-frame case. Two variants were implemented and trained, one using a ConvLSTM cell and one using a ConvGRU cell, both with a 3×3 spatial kernel. The pretrained PFN and backbone were fine-tuned with a reduced learning rate, the detection head was kept frozen for the first epochs, and the recurrent module was trained from scratch.

Results

Neither variant matched the single-frame baseline. The best ConvLSTM checkpoint (epoch 6) reached an NDS of 0.5988 and an mAP of 0.5013, i.e. a regression of -0.018 NDS and -0.013 mAP relative to the baseline. The ConvGRU variant produced numerically similar results and is omitted from the tables for brevity. Qualitative inspection of the predictions revealed stretched and laterally displaced bounding boxes, particularly when the ego vehicle was moving fastest, and a general reduction of recall on small classes. The true-positive metrics also degraded uniformly: the orientation error increased from 0.408 to 0.467 rad and the translation error from 0.308 m to 0.319 m, indicating that the temporal features fed to the head were not just less informative but actively misaligned. Notably, the per-class results (Table 2.2) reveal exceptions to the general regression: the ConvLSTM model improved over the baseline on *motorcycle* (0.489 vs. 0.477), *bicycle* (0.262 vs. 0.205), *construction vehicle* (0.132 vs. 0.129), and *trailer* (0.377 vs. 0.356). These classes share either small size, atypical aspect ratios, or motion patterns that benefit from temporal context across consecutive frames, suggesting that short-term memory is genuinely helpful for them even when the overall head distribution is misaligned.

Failure Analysis

Five interacting factors were identified as responsible for the observed degradation.

Distribution mismatch at the detection head. The pretrained CenterPoint head was optimised on the feature statistics produced by a single-frame backbone applied to one BEV map. The ConvLSTM and ConvGRU layers apply sigmoid/tanh gates and accumulate activations across time, which shifts both the mean and the variance of the BEV feature map fed to the head. The head therefore received an input distribution it had never been trained on, which alone is sufficient to corrupt classification scores and regression offsets even when the underlying temporal information is informative.

Absence of ego-motion compensation. The BEV grids of consecutive keyframes are expressed in different ego coordinate systems, because the ego vehicle moves several metres between captures at 10 Hz. The implementation did not include any explicit warping of past feature maps to the current ego frame, so the recurrent cell was fusing features that, at the same spatial coordinate (i, j), referred to different points in the world. This is precisely the failure mode that TimePillars addresses with a convolutional transformation module guided by an auxiliary task. The stretched boxes observed qualitatively are a direct manifestation of this

misalignment.

Spatial blur from 3×3 temporal convolution on fine voxels. With a voxel size of 0.2 m, a 3×3 ConvLSTM/ConvGRU kernel mixes information across a 1.2 m receptive field at every recurrent step. For pedestrians and traffic cones, this is comparable to or larger than the object itself, so each recurrent step smears the object's features into neighbouring cells. The class-level results in Table 2.2 are broadly consistent with this: the three largest absolute AP drops are barrier (-0.055), truck (-0.044), and traffic cone (-0.042), with pedestrian close behind (-0.040). Bicycle is a notable exception: despite its small footprint, it gained $+0.057$ AP, indicating that distinctive inter-frame motion cues outweigh the blur penalty for this class.

Sweep concatenation in the data loader. The loader was subsequently rewritten to load each sweep as a separate frame in its own ego coordinate system, which is a prerequisite for any meaningful temporal model.

Implications for the Next Experiment

The failure of this first experiment points to three concrete fixes. First, a 3×3 recurrent kernel on a fine BEV grid blurs small objects, so a different temporal mechanism is needed. Second, the pretrained detection head is sensitive to any change in its input features, so the temporal module must add information as a residual rather than replacing the single-frame features. Third, aligning past and present BEV grids cannot be left to a recurrent cell operating cell-by-cell; the correspondence must be handled explicitly. Together these point towards an attention-based memory bank, in the style of ReasonNet [25].

2.1.5 Second Attempt: ReasonNet-Style Temporal Memory Bank

Motivation

ReasonNet [25] introduces a temporal reasoning module that maintains a short-term and a long-term memory bank of BEV features extracted from past frames, and reads from them through a content-based attention mechanism. At each timestep, the current BEV feature map is used to construct a set of queries that attend over the keys stored in the bank, and the retrieved values are fused with the current feature map before being passed to the detection head. Compared to a recurrent cell, this design has three properties that directly address the failure modes of the first experiment. The spatial correspondence between past and present features is established by the query/key dot-product rather than by their grid position, so the absence of ego-motion compensation is, to first order, hidden by the attention mechanism. The output of the fusion can be expressed as a residual update on the current feature map, which keeps the input distribution of the head close to the single-frame statistics it was trained on. And the spatial receptive field of the query encoder can be chosen independently of the temporal mechanism, so the spatial-blur trade-off of the 3×3 recurrent kernel does not apply.

Architecture

The proposed model, denoted *TemporalPointPillar-MB* (memory bank), is built on top of the pretrained single-frame CenterPoint described above. Given a sequence of LiDAR keyframes, each frame is processed by the shared (pretrained) Pillar Feature Net and the 2D backbone, producing a BEV feature map $F_t \in \mathbb{R}^{C\times H\times W}$ with $C = 384$ channels at resolution 128×128.

At each timestep t, a query encoder $\phi_q : \mathbb{R}^C \to \mathbb{R}^D$ produces a query map $Q_t \in \mathbb{R}^{D\times H\times W}$ from F_t, and a key encoder ϕ_k produces, from past frames, a corresponding key map K_τ that is stored in the bank together with the raw feature map F_τ as its value. Both encoders are implemented as a 3×3 convolution followed by two 1×1 convolutions, giving a 5×5 effective receptive field. Features are ℓ_2-normalised along the channel dimension, so the similarity between a query position i and a key position j is bounded:

$$s_{ij} \;=\; \mathrm{softmax}_j(-\,\|Q_t(i) - K_\tau(j)\|_2)\,.$$

The attention is then used to read from the value bank, producing a retrieved feature map R_t, and the fused output is constructed as a residual update,

$$\tilde{F}_t \;=\; F_t \;+\; \mathrm{GN}(W_o\, R_t),$$

where W_o is a 1×1 projection and GN a group normalisation layer. At initialisation W_o is small, so the detection head receives $\tilde{F}_t \approx F_t$ and is expected to reproduce the baseline scores from the first epoch. The memory update itself proceeds in two stages: a short-term bank that keeps the last T_s frames in raw form, and a long-term bank that admits selected positions from the short-term bank according to a usage-based promotion rule similar to the one in ReasonNet [25]. The long-term bank was disabled ($t_l = 0$) for this experiment: only the short-term memory bank was trained and evaluated, and the long-term promotion mechanism was not used.

Results

Despite the residual formulation, the model collapsed during training. The best checkpoint (epoch 8) reached an NDS of 0.0309 and an mAP of 0.0016, more than an order of magnitude below the baseline. The collapse is class-imbalanced: only *car* retained any detection capability, with a per-class AP of 0.0162, and all other nine nuScenes classes dropped to AP exactly 0 (Table 2.2). The true-positive errors saturated at their worst-case values (mATE = 0.956 m, mASE = 0.864, mAOE = 1.192 rad, mAVE = 1.129 m/s, mAAE = 0.880), which in the nuScenes protocol indicates that there are essentially no matched detections to compute meaningful errors on.

Diagnostic Visualisations and Failure Analysis

The collapse was investigated using a debug pipeline that renders, for selected query positions, the spatial distribution of the attention weights they place on the short-term memory bank, together with summary maps of the memory occupancy and the detection-head outputs. Two representative panels are shown in Figures 2.1 and 2.2.

Figure 2.1 reports the full diagnostic suite for a single training step. The top row shows the BEV energy $\|F_t\|$ (left), the post-fusion energy $\|\tilde{F}_t\|$ (centre), and their per-cell absolute difference (right). The fused map preserves the gross spatial structure of the input but the difference map is concentrated in a small number of bright cells at the periphery rather than on the objects in the scene. The middle row shows the predicted heatmap channel `mp_ch0`, the average short-term memory usage `st_usage_mean`, and the mean of `mp_ch0` over the short-term frames (`st_mp0_mean`). The `st_usage_mean` map is essentially uniform across the 128×128 grid, with high-usage cells localised at the margins where there are no objects. The bottom row shows the attention maps from three query positions (cyan crosses); each map is nearly uniform over the entire BEV grid, with the only deviations from uniformity being mass leaked towards the same peripheral cells visible in the usage map.

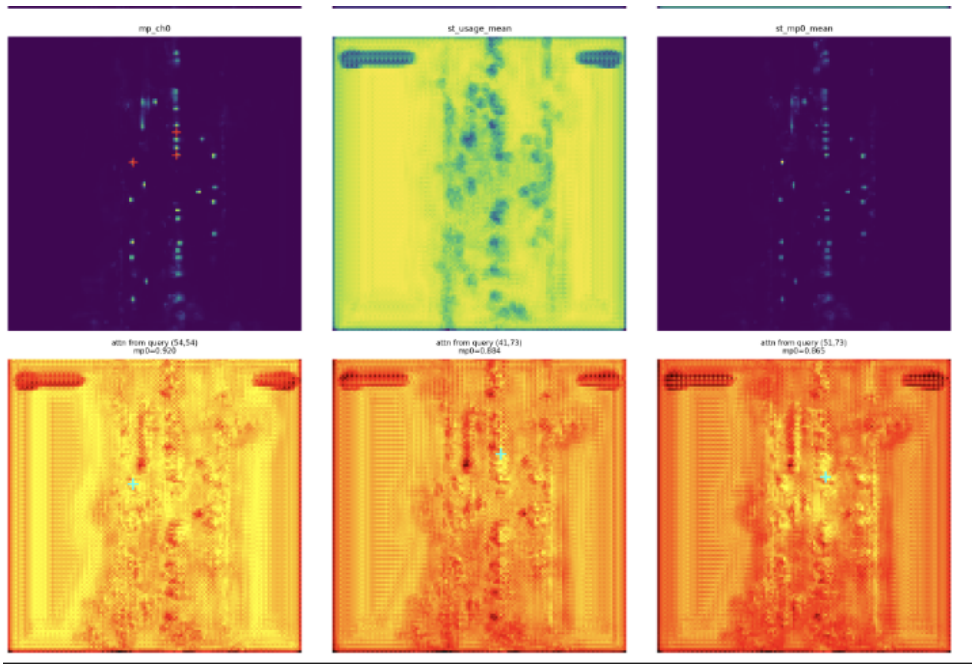


Figure 2.1: Early collapse diagnostic for *TemporalPointPillar-MB* (short-term bank only). Top: BEV energy, post-fusion energy, and per-cell difference $|\tilde{F}_t - F_t|$. Middle: heatmap `mp_ch0`, short-term usage `st_usage_mean`, and `st_mp0_mean`. Bottom: attention maps from three query positions (cyan crosses). Usage is near-uniform and attention concentrates on peripheral cells—the signature of a collapsed memory.

Figure 2.2 shows a later snapshot in which the same three quantities (`mp_ch0`, `st_usage_mean`, `st_mp0_mean`) are reported together with three query-attention maps. The collapse is more ad-

vanced: the attention maps are saturated, with the same dominant cells (top-left and top-right corners) receiving most of the mass from every query position, regardless of where the cyan cross is placed. At this point the memory bank has degenerated into a position-agnostic retriever that returns essentially the same context vector for every query, which is then mixed back into the BEV map through the residual and overwhelms the object signal that the pretrained head expects.

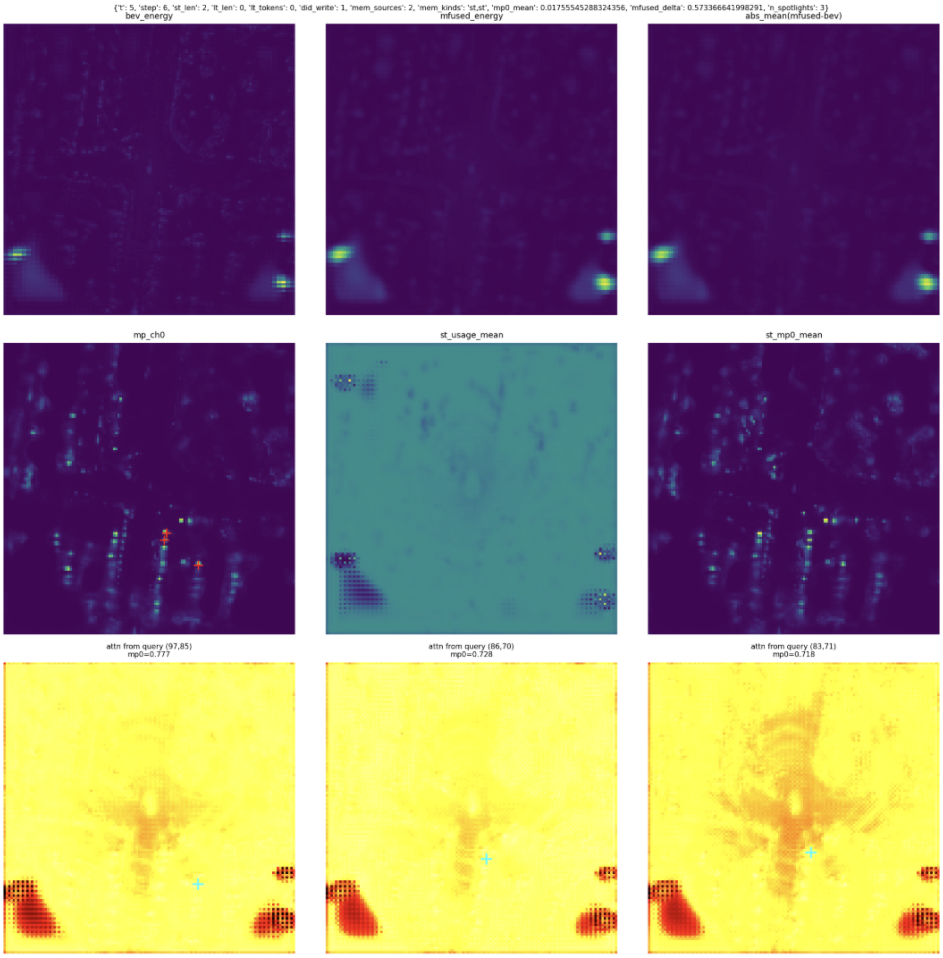


Figure 2.2: Advanced collapse state on a different sample (short-term bank only). Top: `mp_ch0`, `st_usage_mean`, and `st_mp0_mean`. Bottom: attention maps from three query positions (`mp0` values above). Attention saturates at the same corner cells for every query, confirming the bank has become a position-agnostic retriever—consistent with the class collapse in Table 2.2.

Three concrete causes were identified for this collapse.

Insufficient discriminability of the query/key encoder. For the residual update to be useful, queries placed on semantically distinct positions must produce distinct attention patterns. The 5×5 effective receptive field of the query/key encoder is sufficient in principle to distinguish neighbouring objects of the same class, but only if the encoder is trained with a loss that incentivises this discriminability. The detection loss alone does not: any encoder that returns the same query for every position drives the residual towards a constant offset and recovers

a degraded but stable version of the baseline scores, which is a local minimum of the loss landscape. The uniform attention maps in Figures 2.1 and 2.2 are consistent with the model having settled into this local minimum during early training and having then drifted away from it without recovering object-specific attention.

Distance-based similarity in an unbounded feature space. The implementation used a softmax over the negative ℓ_2 distance between ℓ_2-normalised query and key vectors. While this is mathematically well-posed and bounds the distance to $[0, 4]$, in practice the softmax is very flat for any moderate temperature, because all pairs of normalised vectors lie on the unit hypersphere and most distances are concentrated near the mean of the distribution. Without an explicit temperature scaling or a contrastive auxiliary loss, the resulting attention distribution is close to uniform, which is exactly what the diagnostic figures show. This problem is well-known in memory-based segmentation: STCN [36] showed that applying softmax over dot-product similarity causes a small subset of memory entries to dominate all queries regardless of content, collapsing effective memory coverage. STCN's solution was to use the raw (unnormalised) negative squared ℓ_2 distance as the similarity score, which has a much wider dynamic range than the cosine-equivalent used here and does not require temperature tuning. Adopting this formulation, or adding an explicit temperature $\tau \ll 1$ to sharpen the cosine-based softmax, would be a direct fix for the flat-attention failure mode observed in this experiment.

Per-frame normalisation of cumulative attention weights. The long-term promotion rule promotes positions that accumulate the most attention over time. This works correctly when attention is sharp, but softmax always sums to 1 per frame, so a collapsed flat distribution contributes the same total mass as an informative one. Once attention collapses, the promotion rule cannot distinguish useful keys from useless ones and provides no corrective effect [25]. This is why the long-term bank was disabled ($t_l = 0$) for this experiment.

Implications

The second experiment confirms that the failure modes diagnosed in the first experiment cannot be fully addressed by substituting an attention mechanism for the recurrent cell, at least not without a training procedure that explicitly enforces attention discriminability (e.g. contrastive key learning with an InfoNCE-style loss, or ego-motion warping supervision). It also shows that the residual formulation, while necessary, is not sufficient: the collapse path through the local minimum of zero useful residual is too attractive for the detection loss alone to escape.

2.1.6 Quantitative Summary

Tables 2.1 and 2.2 report the overall and per-class metrics on the nuScenes validation split for the three trained models. The ConvLSTM is the best of the attempted temporal variants and is the closest to a publishable ablation point; it also shows selective per-class gains on *motorcycle*,

bicycle, *construction vehicle*, and *trailer*—the classes most likely to benefit from short-term temporal context. The memory-bank result is included because the nature of its failure informs the design choices discussed in the next subsection.

Table 2.1: Overall detection results on the nuScenes validation split. Higher is better for mAP and NDS; lower is better for the five true-positive errors. Numbers are absolute (mAP and NDS as fractions; errors in their native units).

Model	Epoch	mAP ↑	NDS ↑	mATE ↓	mASE ↓	mAOE ↓	mAVE ↓	mAAE ↓
CenterPoint (baseline)	20	0.5146	0.6171	0.308	0.259	0.408	0.234	0.194
ConvLSTM	6	0.5013	0.5988	0.319	0.266	0.467	0.261	0.205
ReasonNet MB	10	0.0016	0.0309	0.956	0.864	1.192	1.129	0.880

Table 2.2: Per-class mean AP on the nuScenes validation split. ConvLSTM improves over the baseline on *motorcycle*, *bicycle*, *construction vehicle*, and *trailer* while regressing on all remaining classes. The ReasonNet MB row shows the class-imbalanced nature of the collapse: only *car* retains any detection capability, while all other classes drop to exactly zero.

Model	car	truck	c.v.	bus	trailer	barrier	motor.	bicycle	ped.	cone
Baseline	0.835	0.520	0.129	0.641	0.356	0.598	0.477	0.205	0.798	0.588
ConvLSTM (ep. 6)	0.818	0.476	0.132	0.612	0.377	0.543	0.489	0.262	0.758	0.546
ReasonNet MB (ep. 8)	0.016	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000

2.1.7 Discussion

The two experiments above produce four pieces of evidence that inform the continuation of this work. First, the OpenPCDet implementation of CenterPoint with a PointPillars encoder is a strong and reproducible single-frame baseline on nuScenes (NDS 0.6171, mAP 0.5146) and is the right starting point for any downstream component. Second, a naive application of the LSTM Pillars / TimePillars recipe [32, 33] regresses NDS by about 0.018 points and does so for reasons that are intrinsic to the recurrent-on-fine-BEV-grid design, not to the specific hyperparameters used. Third, the ReasonNet-style attention bank [25], in the form implemented here, collapses to a position-agnostic retriever and would require a substantially different training objective (contrastive key learning, ego-motion warping supervision, or temperature-scaled similarity) to escape the local minimum identified above. Fourth, and most importantly, both families of temporal extensions are bottlenecked by the same root cause: a pretrained detection head that is sensitive to any perturbation of its input feature distribution, and a temporal mechanism that has to fight this sensitivity rather than exploit it.

2.2 LiViP3D: Query-Based LiDAR–Camera Fusion for End-to-End Perception and Prediction

This section documents *LiViP3D*, a second, independent line of experiments that fuses LiDAR into ViP3D's query-based, fully differentiable perception-and-prediction pipeline using ideas from TransFusion, and reports results from three progressive training stages on nuScenes [27].

2.2.1 Related Work

ViP3D [37] treats detection, tracking and trajectory prediction as one differentiable pipeline: sparse DETR3D-style [38] agent queries cross-attend to multi-view image features, persist across frames through a MOTR-style [39] memory bank, and are decoded into future trajectories by a VectorNet map encoder with a multi-modal regression head. Its End-to-end Prediction Accuracy (EPA) metric penalises both missed and hallucinated agents, making it the natural benchmark for query-based pipelines. ViP3D is camera-only, so its queries must localise agents from image evidence alone – weak at range and under occlusion.

TransFusion [40] addresses the complementary problem of *hard-associated* LiDAR-camera fusion, where each LiDAR point is matched to one image pixel via calibration and is therefore brittle to miscalibration and point sparsity. It replaces this with a two-layer decoder: the first layer predicts boxes from an input-dependent, category-aware heatmap over the LiDAR BEV plane; the second softly attends each query to image features, spatially constrained by a Gaussian mask around the query's projected box (spatially modulated cross attention, SMCA), removing the need for a rigid point-pixel correspondence.

LiViP3D combines the two: it keeps ViP3D's query-as-interface design for tracking and prediction, but replaces its camera-only, input-independent query initialisation with TransFusion's heatmap-based, soft-associated fusion mechanism. The supervisor's subsequent Li-ViP3D [41] and Li-ViP3D++ [42] papers extend this same direction with a learned, query-gated deformable fusion (QGDF) in place of the heatmap/SMCA mechanism used here.

2.2.2 Model Architecture

LiViP3D extends ViP3D's MMDetection3D-based codebase with a LiDAR branch and a TransFusion-style two-layer decoder. Two backbones run per frame: a LiDAR branch (`PillarFeatureNet` → `PointPillarsScatter` → `SECOND` [43] → `SECONDFPN`) producing a 384-channel BEV map from up to 10 accumulated sweeps, and an image branch (ResNet-50, DCNv2 in the last two stages, + FPN, 4 levels × 256 channels) over the 6 surround cameras. Track state persists across frames in an `Instances` object (query, reference point, object id), governed by a runtime tracker (confirm/keep/delete thresholds 0.4/0.35, 5-frame miss tolerance).

Heatmap-based query initialisation. Unlike ViP3D's fixed, input-independent queries,

LiViP3D initialises queries from a class-specific heatmap predicted over the LiDAR BEV map (TransFusion Sec. 3.2): local-maxima peaks become query positions/features, summed with a category embedding, so initialisation is input-dependent and category-aware from stage 1 onward. From stage 2, an *image-guided* extension adds a second heatmap: each camera's finest feature level is height-pooled to columns and cross-attended into BEV space (one attention module per camera, averaged), scored into an image heatmap, and simply averaged with the LiDAR heatmap, $\hat{S}_{LC} = (\hat{S} + \hat{S}_{img})/2$ – the same idea as TransFusion Sec. 3.6, though implemented with dense per-camera attention rather than frustum-restricted attention.

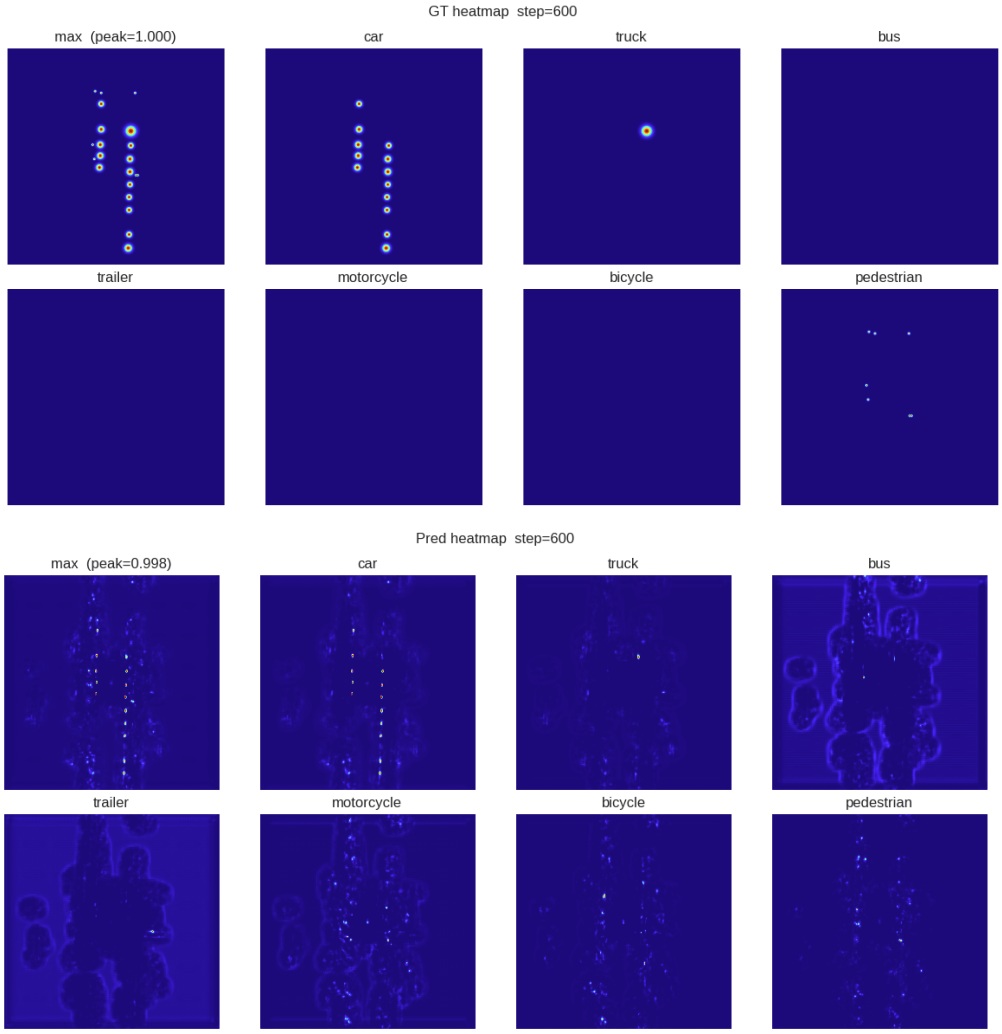


Figure 2.3: Ground-truth (top) and predicted (bottom) class-specific heatmap $\hat{S}$ over the LiDAR BEV plane, stage 1 (LiDAR only), training step 600. Each panel shows the aggregate max channel plus one channel per class; probability mass is already concentrated on *car* and *pedestrian*, consistent with the per-class EPA/AMOTA breakdown in Tables 2.3 and 2.4.

Two-layer decoder. The first layer cross-attends queries to the LiDAR BEV map at their reference point. The second layer, active from stage 3, is SMCA: an anisotropic Gaussian mask sized from each query's projected box extent is added log-additively to the image cross-

attention logits before softmax (TransFusion Sec. 3.4), computed per camera and averaged over the cameras where the query is visible, with a learned fallback for queries visible in none. Below stage 3 this layer is skipped.

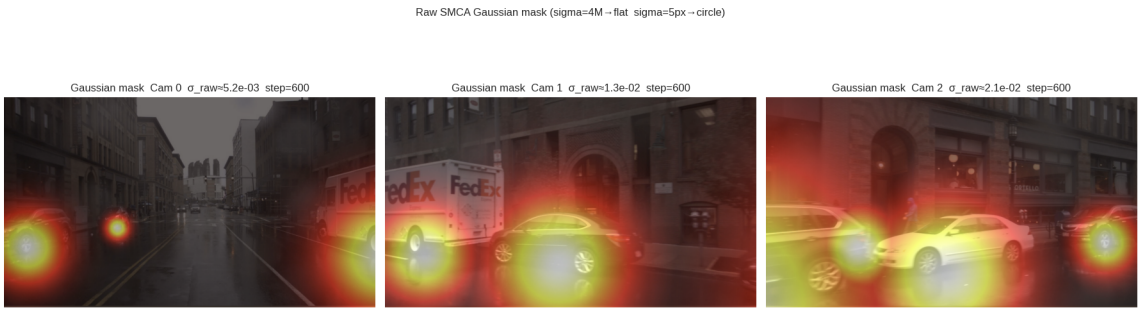


Figure 2.4: Raw SMCA Gaussian mask for three of the six cameras, training step 600, overlaid on the input image. The mask is anisotropic and camera-dependent (σ_{raw} shown per camera), sized from the projected 3D box extent as in TransFusion Sec. 3.4.



Figure 2.5: SMCA cross-attention for the top-20 heatmap queries, same cameras and step as Fig. 2.4. Top: attention after the Gaussian mask is applied, concentrated tightly on nearby vehicles. Bottom: the same attention with the mask removed ("pure attention"), visibly noisier and less object-centred – illustrating why the Gaussian prior helps the network attend to the correct region early in training.

Tracking and prediction. Cross-frame association reuses ViP3D's mechanisms unchanged: a MOTR-style FIFO memory bank (depth 4) refreshes each track's query via temporal

cross-attention, and a VectorNet map encoder feeds a 6-mode regression decoder (12-step horizon, winner-take-all variety loss) that turns each tracked query directly into trajectories. A designed but currently disabled appearance-based past-motion refinement module also exists in the codebase (`apr_cfg=None` in every run reported here) and is left for future work.

2.2.3 Experimental Setup

All models train on nuScenes: 6 surround cameras resized to 960×544 px, up to 10 accumulated LiDAR sweeps, and a detection range of ± 51.2 m in x/y and $[-5, 3]$ m in z. The LiDAR branch voxelises this range into $0.2 \times 0.2 \times 8$ m pillars (up to 20 points per pillar, 30k training / 40k inference voxels), scattered into a 512×512 BEV grid and reduced by `SECOND/SECONDFPN` to the 384-channel feature map used throughout the decoder; 300 object queries are used in every stage. Optimisation uses AdamW (lr 2×10^{-4}, weight decay 0.01, gradient clipping at norm 35) with a cosine schedule and a 500-iteration linear warmup. The image backbone is frozen throughout every LiViP3D stage (lr_mult = 0, unlike the baseline's lightly fine-tuned camera path), so detection quality in stages 1–2 rests on the LiDAR branch alone; stage 3 additionally freezes the LiDAR branch and fine-tunes only the fusion layers from stage 2's checkpoint.

2.2.4 Results

Table 2.3 reports the best available checkpoint per run, using the repository's own `-max` label where present and otherwise the epoch with the best combined detection/prediction quality. The evaluation pipeline is tracking-based, so AMOTA and Recall replace mAP/NDS; mADE, mFDE, MR and EPA follow ViP3D's protocol. The shaded columns break EPA down per class: the aggregate figure is driven almost entirely by *car* and *pedestrian*, with *bus*, *trailer* and *motorcycle* still at or near zero.

Table 2.3: LiViP3D results on the nuScenes validation split. AMOTA/Rec. are nuScenes tracking metrics; mADE, mFDE, MR and EPA (minADE, minFDE, Miss Rate, End-to-end Prediction Accuracy) follow ViP3D's protocol. Shaded columns: per-class EPA.

								Per-class EPA ↑						
Experiment	Ep.	AMOTA ↑	Rec. ↑	mADE ↓	mFDE ↓	MR ↓	EPA ↑	Car	Truck	Bus	Trail.	Moto.	Bike	Ped.
ViP3D (baseline)	16	0.209	0.350	1.644	2.635	0.267	0.136	0.129	0.035	0.056	0.000	0.152	0.168	0.130
S1 (LiDAR only)	12	0.108	0.145	1.362	2.378	0.292	0.137	0.180	0.069	0.000	0.000	0.000	0.000	0.068
S2 (+ img-guided)	10	0.083	0.110	1.067	1.987	0.199	0.223	0.263	0.042	0.000	0.000	0.000	0.000	0.181
S3 (+ SMCA, 3 ep)	3	0.121	0.165	1.153	2.256	0.220	0.228	0.268	0.078	0.000	0.000	0.004	0.017	0.199
S3 (+ SMCA, 6 ep)	5	0.109	0.148	1.040	1.886	0.208	0.248	0.303	0.076	0.000	0.000	0.000	0.000	0.219

Table 2.4 breaks the same five runs down into the raw matched-track counts underlying AMOTA, plus per-class AMOTA (shaded). Every LiViP3D stage collapses to exactly zero on *bus*, *trailer*, *motorcycle* and *bicycle* at these checkpoints, with only *car*, *truck* and *pedestrian*

contributing – unlike the camera-only baseline, which retains non-zero AMOTA across all seven classes.

Table 2.4: Raw nuScenes tracking counts underlying the AMOTA column of Table 2.3: matched true positives (TP), false positives (FP) and false negatives (FN), summed over the validation split. Shaded columns: per-class AMOTA.

						Per-class AMOTA ↑						
Experiment	Ep.	AMOTA ↑	TP ↑	FP ↓	FN ↓	Car	Truck	Bus	Trail.	Moto.	Bike	Ped.
ViP3D (baseline)	16	0.209	45497	15364	55159	0.410	0.129	0.179	0.005	0.200	0.219	0.321
S1 (LiDAR only)	12	0.108	37549	8992	63885	0.352	0.029	0.000	0.000	0.000	0.000	0.377
S2 (+ img-guided)	10	0.083	33208	6325	68085	0.328	0.000	0.000	0.000	0.000	0.000	0.254
S3 (+ SMCA, 3 ep)	3	0.121	42385	9845	58796	0.375	0.034	0.000	0.000	0.000	0.000	0.435
S3 (+ SMCA, 6 ep)	5	0.109	38332	6732	62929	0.389	0.091	0.000	0.000	0.000	0.000	0.282

Table 2.5 adds inference efficiency – parameter count and measured latency/FPS on a single RTX 4090.

Table 2.5: Inference efficiency of the five checkpoints on a single NVIDIA RTX 4090 (batch size 1, 200-frame average after 30 warmup frames).

Experiment	AMOTA ↑	Params (M) ↓	Latency (ms) ↓	FPS ↑
ViP3D (baseline)	0.209	41.77	425.2	2.35
S1 (LiDAR only)	0.108	41.60	230.3	4.34
S2 (+ img-guided)	0.083	43.85	520.3	1.92
S3 (+ SMCA, 3 ep)	0.121	43.85	579.0	1.73
S3 (+ SMCA, 6 ep)	0.109	43.85	607.5	1.65

2.2.5 Discussion

Two patterns stand out. First, tracking quality (AMOTA) is highest for the camera-only ViP3D baseline and lowest for the early LiViP3D stages, while prediction quality (EPA, mADE) improves monotonically from stage 1 through stage 3. This tracks the training budget rather than a fusion failure: the baseline fine-tunes its image backbone for 24 epochs, whereas every LiViP3D stage freezes it and gives the LiDAR branch only a fraction of that budget (12/20, then 10/15 epochs) before responsibility shifts to the fusion layers – the metric the curriculum actually targets keeps improving even though raw tracking recall has not yet caught up.

Second, fusion carries a real latency cost that the detection/prediction metrics alone do not show (Table 2.5): stage 2 and stage 3 add only ~2M parameters over the baseline (43.85M vs. 41.77M) yet run at less than half its frame rate (1.65–1.92 vs. 2.35 FPS), since every frame now runs both backbones plus the image-guided BEV projection and, from stage 3, SMCA cross-attention; LiDAR-only stage 1 is both the lightest model and by far the fastest

(4.34 FPS). Equalising epoch budgets across stages for a cleaner comparison, running the follow-up data-augmentation experiments across all three stages, and evaluating the currently dormant appearance-refinement module are the natural next steps.

References

[1] Y. Huang, J. Du, Z. Yang, Z. Zhou, L. Zhang, and H. Chen, "A survey on trajectory-prediction methods for autonomous driving," *IEEE Transactions on Intelligent Vehicles*, vol. 7, no. 3, pp. 652–674, 2022.

[2] S. Ettinger, S. Cheng, B. Caine, C. Sun, and et al., "The Waymo open motion dataset: A large-scale benchmark for motion forecasting," in *Proceedings of the Conference on Robot Learning (CoRL)*, 2021.

[3] Nay and et al., "Wayformer: Motion forecasting via simple and efficient attention networks," *arXiv preprint arXiv:2207.05844*, 2022. [Online]. Available: https://arxiv.org/abs/2207.05844

[4] *Road vehicles — Safety of the intended functionality (SOTIF)*, International Organization for Standardization Std. ISO 21 448:2022, 2022.

[5] N. A. Madjid, A. Ahmad, M. Mebrahtu, Y. Babaa, A. Nasser, S. Malik, B. Hassan, N. Werghi, J. Dias, and M. Khonji, "Trajectory prediction for autonomous driving: Progress, limitations, and future directions," *arXiv preprint arXiv:2503.03262*, 2025. [Online]. Available: https://arxiv.org/abs/2503.03262

[6] "VectorNet: Encoding HD maps and agent dynamics from vectorized representation," arXiv preprint, 2020. [Online]. Available: https://arxiv.org/abs/2005.04259

[7] H. Zhao, J. Gao, T. Lan, C. Sun, B. Sapp, B. Varadarajan, Y. Shen, Y. Shen, Y. Chai, C. Schmid, C. Li, and D. Anguelov, "TNT: Target-driven trajectory prediction," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2020, workshop paper; goal-conditioned forecasting.

[8] J. Gu, C. Sun, and H. Zhao, "DenseTNT: End-to-end trajectory prediction from dense goal sets," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[9] Li *et al.*, "BEVerse: Unified perception and prediction in camera-centric BEV," *arXiv preprint arXiv:2205.09743*, 2022. [Online]. Available: https://arxiv.org/abs/2205.09743

[10] J. Hu, J. F. M. Zhang, P. H. S. Torr *et al.*, "FIERY: Future instance prediction in Bird's-Eye view from surround monocular cameras," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[11] Djuric *et al.*, "MultiXNet: Multiclass multistage multimodal motion forecasting," *arXiv preprint arXiv:2006.02000*, 2020. [Online]. Available: https://arxiv.org/abs/2006.02000

[12] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.

[13] E. A. Wan and R. van der Merwe, "The unscented kalman filter," in *Proc. IEEE Adaptive Systems for Signal Processing, Communications, and Control Symposium*, 2000, pp. 153–158.

[14] N. J. Gordon, D. J. Salmond, and A. F. M. Smith, "Novel approach to nonlinear/non-Gaussian Bayesian state estimation," *IEE Proceedings F (Radar and Signal Processing)*, vol. 140, no. 2, pp. 107–113, 1993.

[15] L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257–286, 1989.

[16] D. Helbing and P. Molnar, "Social force model for pedestrian dynamics," *Physical Review E*, vol. 51, no. 5, pp. 4282–4286, 1995.

[17] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*. MIT Press, 2006.

[18] A. Alahi, K. Goel, V. Ramanathan, A. Robicquet, L. Fei-Fei, and S. Savarese, "Social LSTM: Human trajectory prediction in crowded spaces," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 961–971.

[19] A. Gupta, J. Johnson, L. Fei-Fei, S. Savarese, and A. Alahi, "Social GAN: Socially acceptable trajectories with generative adversarial networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2255–2264.

[20] M. Liang, B. Yang, R. Hu, Y. Zhou, S. Fidler, and R. Urtasun, "Learning lane graph representations for motion forecasting," in *European Conference on Computer Vision (ECCV)*, 2020.

[21] T. Phan-Minh, E. C. Grigore, F. A. Boulton, O. Beijbom, and E. M. Wolff, "CoverNet: Multimodal behavior prediction using trajectory sets," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

[22] T. Salzmann, B. Ivanovic, P. Chakravarty, and M. Pavone, "Trajectron++: Dynamically-feasible trajectory forecasting with heterogeneous data," in *European Conference on Computer Vision (ECCV)*, 2020.

[23] J. Ngiam, B. Caine, V. Vasudevan, Z. Zhang, H. L. Chiang, J. Ling, R. Roelofs, A. Bewley, C. Liu, A. Venugopal, D. Weiss, B. Sapp, Z. Chen, and J. Shlens, "Scene transformer: A unified architecture for predicting multiple agent trajectories," in *International Conference on Learning Representations (ICLR)*, 2021. [Online]. Available: https://openreview.net/forum?id=KJk_H1To6A

[24] B. Philion and S. Fidler, "Lift, splat, shoot: Encoding images from arbitrary camera rigs by implicitly unprojecting to Bird's-Eye view," in *European Conference on Computer Vision (ECCV)*, 2020.

[25] H. Shao, L. Wang, R. Chen, S. L. Waslander, H. Li, and Y. Liu, "Reasonnet: End-to-end driving with temporal and global reasoning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[26] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? the KITTI vision benchmark suite," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2012, pp. 3354–3361.

[27] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, "nuscenes: A multimodal dataset for autonomous driving," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

[28] M.-F. Chang, J. Lambert, P. Sangkloy, J. Singh, S. Bak, A. Hartnett, D. Wang, P. Carr, S. Lucey, D. Ramanan, and J. Hays, "Argoverse: 3d tracking and forecasting with rich maps," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[29] R. Kesten, M. Usman, J. Houston, T. Pandya, and et al., "Level 5 AV dataset 2019," in *Proc. of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2019.

[30] X. Huang, P. Wang, X. Cheng, D. Zhou, Q. Geng, and R. Yang, "ApolloScape: 3d car understanding from monocular images," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2018.

[31] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, "Pointpillars: Fast encoders for object detection from point clouds," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[32] R. Huang, W. Zhang, A. Kundu, C. Pantofaru, D. A. Ross, T. Funkhouser, and A. Fathi, "An LSTM approach to temporal 3D object detection in LiDAR point clouds," in *European Conference on Computer Vision (ECCV)*, 2020.

[33] E. L. Calvo, B. Taveira, F. Kahl, N. Gustafsson, J. Larsson, and A. Tonderski, "Timepillars: Temporally-recurrent 3D LiDAR object detection," 2024.

[34] T. Yin, X. Zhou, and P. Krähenbühl, "Center-based 3D object detection and tracking," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.

[35] OpenPCDet Development Team, "OpenPCDet: An open-source toolbox for 3D object detection from point clouds," https://github.com/open-mmlab/OpenPCDet, 2020.

[36] H. K. Cheng, Y.-W. Tai, and C.-K. Tang, "Rethinking space-time networks with improved memory coverage for efficient video object segmentation," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[37] J. Gu, C. Hu, T. Zhang, X. Chen, Y. Wang, Y. Wang, and H. Zhao, "ViP3D: End-to-end visual trajectory prediction via 3D agent queries," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[38] Y. Wang, V. C. Guizilini, T. Zhang, Y. Wang, H. Zhao, and J. Solomon, "DETR3D: 3D object detection from multi-view images via 3D-to-2D queries," in *5th Annual Conference on Robot Learning (CoRL)*, 2021.

[39] F. Zeng, B. Dong, Y. Zhang, T. Wang, X. Zhang, and Y. Wei, "MOTR: End-to-end multiple-object tracking with transformer," in *European Conference on Computer Vision (ECCV)*, 2022.

[40] X. Bai, Z. Hu, X. Zhu, Q. Huang, Y. Chen, H. Fu, and C.-L. Tai, "TransFusion: Robust LiDAR-camera fusion for 3D object detection with transformers," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[41] M. Halinkovic, A. Vinel, and W. Benesova, "Li-viP3D: Enhancing end-to-end perception and prediction with camera-LiDAR fusion," in *2025 IEEE International Automated Vehicle Validation Conference (IAVVC)*, 2025, pp. 1–6.

[42] M. Halinkovic, N. Masarykova, A. Vinel, and M. Galinski, "Li-viP3D++: Query-gated deformable camera–LiDAR fusion for end-to-end perception and trajectory prediction," 2026.

[43] Y. Yan, Y. Mao, and B. Li, "SECOND: Sparsely embedded convolutional detection," *Sensors*, vol. 18, no. 10, p. 3337, 2018.